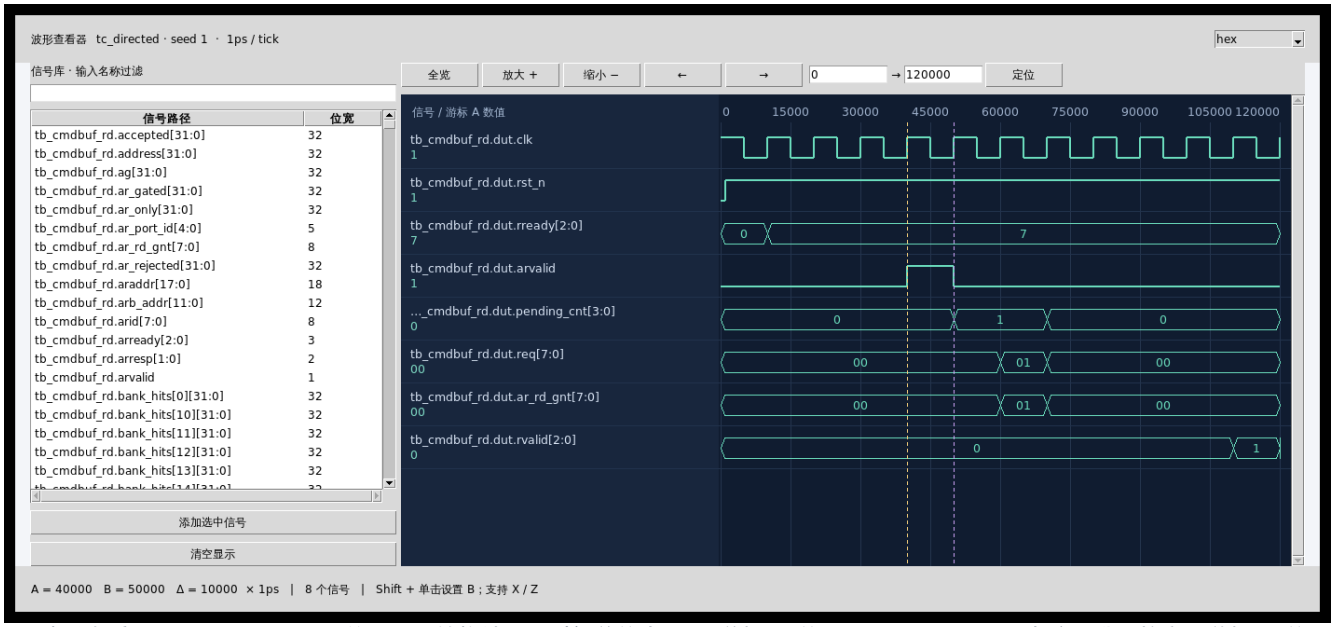
端口激励与时钟上升沿对齐

说明 47 / 2026-11-13 / CMDBUF Studio

main_phase 的 rready、AR、grant、Bank 状态和 MEM 返回在时钟上升沿的同一仿真时间点更新。DUT 先用沿前值完成本拍采样，随后环境根据更新后的反馈准备下一拍。默认 10 ns 产生第一个工作上升沿，cycle=0 在 10 ns 驱动，20 ns 被采样。

时间点	DUT 与检查器	main_phase 驱动
0 ns	clk 初始为 1，rst_n 为 0	初始化接口
1 ns	异步复位释放	业务输入保持初值
10 ns	采样初始输入并更新 DUT	在同一时间点驱动 cycle=0
20 ns	采样 cycle=0 的输入	在同一时间点驱动 cycle=1

真实 VCS 波形



Tk 波形查看器显示 tc_directed 的 FSDB 转换结果，时间单位为 ps。游标 A 位于 40 ns，arvalid 在该上升沿拉高。游标 B 位于 50 ns，DUT 用沿前的 arvalid 接收 AR，随后 arvalid 拉低，pending_cnt 更新。

在 Verdi 查看同一用例

```
make run CASE=tc_directed SEED=1 WAVES=1
make wave REGRESSION=latest CASE=tc_directed SEED=1
```

基准零延时 RTL 的业务输入和输出均与上升沿对齐。1 ns 释放复位时可看到复位及其组合反馈变化。接入带传播延时的 RTL 或门级模型时，输出按该模型的实际时序显示。

main_phase 的驱动顺序

新版模板把事件等待封装在 begin_cycle() 和 sample_context()。用例仍用普通赋值和 if / else 约束输入。下面是对齐相关的骨架，AR、grant、Bank 与 MEM 赋值沿用相应用例模板。

```
for (int unsigned cycle=0; cycle<num_cycles; cycle++) begin
    @(posedge clk);
    begin_cycle(cycle);
    rst_n = 1;
    rready = 3'b111;
    clk_low_ns = 5; clk_high_ns = 5; clk_pause_ns = 0;
    drive_control();
    sample_context();
    // Assign AR, grants, Bank status, and MEM inputs.
    drive_inputs();
end
```

begin_cycle() 等待本沿 DUT 寄存器更新。drive_control() 写入 rready，sample_context() 在同一时间点等待反馈稳定，随后用例按 arready、req 和 MEM 日历赋值。drive_inputs() 校验后驱动。原生 SVA 和事务观察器继续使用沿前采样。

旧模板和归档

旧模板在 @(posedge clk) 后和 drive_control() 后的固定 #1 等待由编译副本迁移。源用例可直接继续使用，手动维护模板时可以删去这两处 #1。make replay 使用原归档的二进制与驱动偏移。需要对齐波形时用新版 make run 或 make regress 生成。

定位和核对

assigned.csv 的 time_ps 记录驱动时间，等于 launch_ps。expected_sample_ps 是下一上升沿。samples.csv 记录 DUT 在下一沿实际消耗的输入。FSDB 转 Formal 使用沿前值，所以生成的 assume 与 assert 对应采样拍。

VCS 验收

定向、随机和停钟用例共核对 4967 次业务端口跳变，均落在上升沿。make check-test 通过 162 项检查。新版随机 FSDB 的 276 个沿前快照与观察器逐拍一致。同一 seed 42 的 2000 拍严格模式运行，采样、事务和时钟复位记录与调整前逐字节一致。

验收记录位于 verification/posedge_alignment_20261113/。